East-West Virtual Machine Performance

Stephen Hemminger

Microsoft Corp.
stephen@networkplumber.org

Abstract

This paper looks at the theoretical and measured performance of East/West networking using KVM and virtio. The default performance of TCP communications between VM's was 61% of the theoretical maximum. The performance was not improved by changes in configuration such as: changes to segmentation offload, queue disciplines or TCP congestion control. Further testing is needed to examine performance with DPDK, Unikernel, and copy-less I/O.

Keywords

Virtualization, networking, virtio, TCP/IP

Introduction

The current age of cloud computing has changed many applications from being deployed on dedicated hardware to instead being on virtual machines. This has changed the network traffic patterns to increasingly use internal (East/West) virtual networking. The popular architectural models used by projects such as Openstack, OPNFV, and Docker containers all have one or more internal Linux bridges (or OpenVswitch) for inter-VM communications.

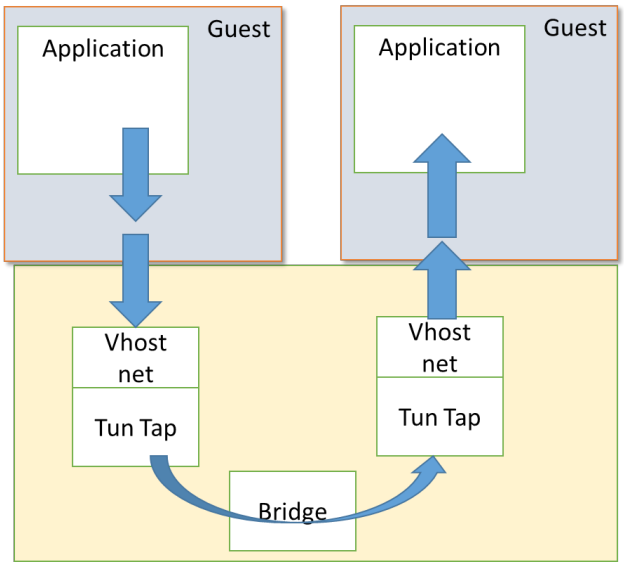


Figure 1: East – West data transfer

Performance measurements

The internal Linux bridge is often blamed for the slow performance of these connections. But the real cause is the overhead of data copies and machine state transitions. A test environment was create to measure these effects.

The test server system had:

- Intel Xeon E5-1620 v4 Broadwell-EP 3.5 Ghz
- Memory DDR4 with 4 banks, effective bandwidth is 52000 MB/sec [1].
- Host KVM with 4.4 kernel (Ubuntu LTS)
- Guest Ubuntu and Debian guest

The maximum possible bandwidth can be determined by the sojourn time for a full size TSO packet.

- Memcpy = 1.2µs
- System call overhead = .5µs
- VM Guest to host transition = .5µs [2]

The total of the 4 memory copies, 2 system calls and 4 VM exits (include at least one TCP ack) is 7.8µs for each 64K bytes of data. This results in a maximum of 128,205 packets/sec which is 67.2 Gbits/sec for inter-VM bandwidth

Measured

This was then compared to actual measurements using iperf3 [3]. Linux supports many different offload options, and these were measured. The basic single stream test performance was:

Parameters	**Throughput**
Default (GRO + GSO)	41 Gbit/sec
Disable Checksum offload	7 Gbit/sec
Disable GRO or GSO	41 Gbit/sec (no change)
Transmit no cache copy	34 Gbit/sec
Rx Busy Poll	Kernel hang

Table 1: Relative performance of different offload options

The Transmit no cache copy option is an experimental value that is normally disabled. In theory it should preserve the CPU cache from getting perturbed by the data copy, but as results show it does not help.

The "Rx Busy poll" is an experimental configuration option that allows kernel to poll for packets. It is intended to reduce the receive latency. Unfortunately with this kind of bulk traffic, the kernel never leaves the busy polling state and triggers internal checks for hung/stuck processes.

Alternatives

Several other alternatives are worth investigating to see if they improve the performance

Single Root I/O Virtualization

Most of the high speed network interface cards (10G and above) support Single Root I/O Virtualization [4]. Although these devices are normally connected to an external port, it is possible to transfer data between Virtual Function devices passed to a Guest VM. The performance of this setup could in theory be faster because there is no any data copies by the host CPU. When the experiment was run using Intel X520 10 Gbit adapters, the VM to VM performance was 17Gbit/sec. This is is higher than the actual NIC speed (10Gbit/sec) but is limited by the PCI-e bandwidth to the device.

TCP Congestion Control

In order to investigate the effects of latency and TCP congestion control the Flexible Network Test (FLENT) [5] framework was used. This test framework combines multiple upload and download streams as well as latency measurements.

Using the default Linux TCP congestion control (CUBIC) [6], the multistream performance was close to the single stream performance of previous tests.

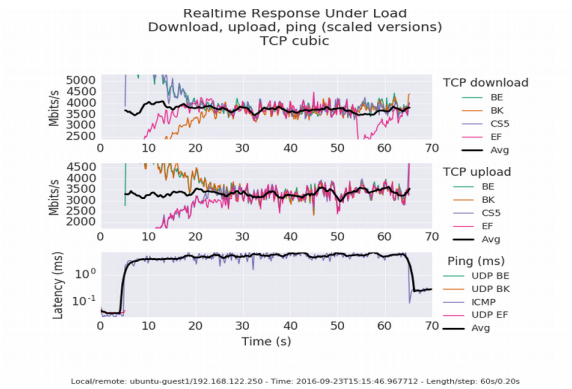


Figure 2: TCP Cubic Realtime Response Under Load test

The Linux kernel supports many different options for TCP congestion control. Of particular interest are the TCP Data Center TCP [7] and Bottleneck Bandwidth and Round-trip (BBR)[8] which avoid packet loss. Testing revealed the same results for the common loss based congestion controls (Reno, BIC, CUBIC) and degraded performance for RTT based congestion control algorithms (Vegas, New Vegas, Common Delay Gradient). The recently released Linux implementation of BBR had unbalanced performance between directions which would make it unsuitable.

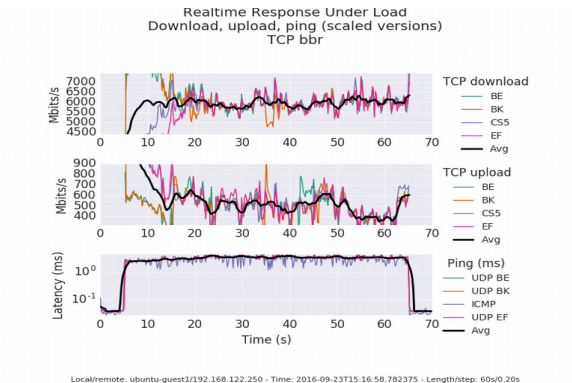


Figure 3: TCP BBR Realtime Response Under Load

Maximum Transmission Unit metrics

The current model of Maximum Transmission Unit in Linux is that the interface MTU is the upper bound. By using TCP path metrics and MTU discovery it should be possible to configure the system so that 64K packets can be passed between VM's while allowing any outbound (North South) traffic to be segmented as needed. This is complex to setup and has issues because the Linux bridge is a pure Layer 2 bridge and will not do Path MTU discovery.

Userspace networking

The copy between user and kernel space which is 30% of the overhead can be eliminated by avoiding the Linux kernel networking stack. This was originally proposed by Van Jacobsen in his network channels presentation[9]. A current project, Transport Layer Development Kit (TLDK) [10] will use the Dataplane Development Kit (DPDK) to access hardware and virtual network devices directly.

A similar alternative architecture is to eliminate the concept of operating system kernel completely in the Guest as is done in Unikernel[11] environments.

Virtio 1.1

These tests exposed performance issues in virtio which may already be addressed by the new Virtio 1.1 specification [12]. This version changes the descriptor layout which reduces the CPU cache misses which are one of the limiting factors in this test.

Conclusions

East/West networking performance is limited by memory bandwidth and virtual machine exit overhead. Future work in batching and offloads may provide incremental performance improvements but large gains will require alternative architectures.

References

1. “DDR3 vs. DDR4: Four Generations, Raw Bandwidth by the Numbers”. Accessed September 20, 2016, http://www.corsair.com/en/blog/2015/september/ddr3_vs_ddr4_generational

2. Grigory R. (2014 Sept 29) “Measurements of Latency for Intel VMX Root — Non-root Transitions,” Accessed September 20, 2016, http://atakua.doesntexist.org/wordpress/2014/09/29/measuring-vmx-root-non-root-transitions/

3. “iPerf - The ultimate speed test tool for TCP, UDP and SCTP”, Accessed October 1, 2016, https://iperf.fr/

4. Y. Dong, X. Yang, J. Li, G. Liao, K. Tian, & H. Guan (2012) “High performance network virtualization with SR-IOV,” *Journal of Parallel and Distributed Computing*. 72 (2012) 1471-1480

5. T. Høiland-Jørgensen, “Flent: The FLExible Network Tester,” Accessed September 23, 2016, https://flent.org/

6. S. Ha, I. Rhee, L. Xu, “CUBIC: A New TCP-Friendly High-Speed TCP Variant,” *ACM SIGOPS*, Vol 42 No. 5 July 2008 64-74

7. M. Alizadeh, A. Greenberg, D. Maltz, J. Padhye, P. Patel, B. Prabhakar, S. Sengupta, M. Sridharan, “Data Center TCP (DCTCP)”, Proceedings of ACM SIGCOMM 2010, 63-74

8. N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, V. Jacobson, “BBR: Congestion-Based Congestion Control,” *ACM Queue*, Vol. 14 No. 5, September-October 2016

9. J. Corbet, “Van Jacobson's network channels,” Linux Conf Au, January 2006, Accessed from https://lwn.net/Articles/169961/

10. “Transport Layer Development Kit,” Accessed October 3, 2016 https://wiki.fd.io/view/TLDK

11. Unikernel. (2016, August 24). In Wikipedia, The Free Encyclopedia. Retrieved 06:02, October 4, 2016, from https://en.wikipedia.org/w/index.php?title=Unikernel&oldid=736047435

12. Tsirkin M., “Quo Vardis Virtio”. *KVM Forum 2016* Accessed September 24, 2016 from https://youtu.be/5QIE0F7nU3U